Agent Modes and Tools

Module 3 · Day 1 (Hands-On + Concept)

Cursor Training Program · ~60 min

Module Overview

Aspect	Details
Duration	~60 minutes
Format	Hands-on exercise + concept
Prerequisites	Module 2 completed, live web app available (or sample provided)
Module Goal	Master different agent modes and the core tools that make agents powerful

Learning Objectives

By the end of this module, participants will be able to:

- Choose between Ask Mode and Agent Mode based on task and safety needs
- Use the Browser Tool to inspect live pages and read console output
- Run terminal commands through the agent and diagnose failures
- Write effective, constrained prompts that avoid scope creep

Agenda

Lesson	Topic	Time
3.1	Ask Mode vs. Agent Mode	18 min
3.2	Browser Tool	18 min
3.3	Terminal Tool	20 min
3.4	Effective Prompting in Practice	22 min

Lesson 3.1

Ask Mode vs. Agent Mode

Concept · 10 min · Exercise · 8 min

The Core Distinction

Aspect	Ask Mode	Agent Mode
Can read files	✅ Yes (with @mentions)	✅ Yes
Can edit files	❌ No	✅ Yes
Can run terminal	❌ No	✅ Yes
Can browse web	❌ No (limited)	✅ Yes (with tool)
Can call tools	❌ No	✅ Yes
Safety level	Very high (read-only)	Moderate (needs oversight)
Best for	Questions, learning, code review	Implementation, debugging, automation

When to Use Each Mode

USE ASK MODE when:

- You have a question about code · Exploring a codebase
- You want a second opinion on design
- You're not ready to make changes · Production environment

USE AGENT MODE when:

- You want the AI to write/change code
- You need to run and react to commands
- Multi-step tasks · Development environment
- You're prepared to review changes

Safety Implications

Risk	Ask Mode	Agent Mode
Unintended code changes	None	Moderate (requires review)
File deletion	None	Possible (needs approval)
Malicious commands	None	Possible (needs approval)
Data leakage	Low	Medium (can read files)
API cost	Low (no tool calls)	Higher (multiple tool calls)

The Mode Continuum

> *"Not every AI interaction needs full agent capabilities."*

Exercise 3.1 — Steps 1–2

Step 1: Open Agent panel (`Cmd+I` / `Ctrl+I`) — note mode indicator at bottom

Step 2: Try to make a change in **Ask Mode**:

```
Change the variable name 'temp' to 'temperature' in the current file.
```

Expected: Agent explains it cannot edit in Ask Mode — offers to show the change as a code block instead.

Exercise 3.1 — Steps 3–5

Step 3: Ask a question Ask Mode handles well:

```
Explain the purpose of the main() function in this file.
What edge cases does it handle?
```

Step 4: Switch to **Agent Mode** via the dropdown

Step 5: Repeat the rename request — agent shows diff for approval

Exercise 3.1 — Step 6 & Success Criteria

Step 6: Practice mode-switching mid-conversation:

```
# Start in Ask Mode: What does this function return?
# Then: "Switch to Agent Mode and fix the off-by-one error"
```

Success Criteria:

- Used Ask Mode for questions · Observed Ask Mode cannot make changes
- Switched to Agent Mode · Made a change with diff review

Lesson 3.2

Browser Tool

Concept · 8 min · Exercise · 10 min

What the Browser Tool Can Do

- Navigate to URLs · Read page content and DOM structure
- See console logs and errors · Take screenshots (depending on model)
- Click elements and interact with pages
- Extract data from live pages

> *"See what your app actually looks like in a browser — not just the source code."*

Browser Tool: With vs. Without

Scenario	Without Browser	With Browser
"Why is the button not showing?"	Guesses from CSS	Sees the rendered page
"Is the API returning data?"	Checks code	Sees network tab
"What console errors?"	Asks you	Reads console directly
"Does responsive layout work?"	Trusts CSS	Views at different sizes

Exercise 3.2 — Steps 1–2

Step 1: Start a local web app (or use a public test page)

```
python -m http.server 8000
# Or use a public test page
```

Step 2: In Agent Mode:

```
Use the browser tool to open http://localhost:8000
Tell me what you see on the page.
```

Exercise 3.2 — Steps 3–4

Step 3: Find specific elements:

```
On that same page, find:
1. The main heading text
2. The number of buttons
3. Any error messages visible
```

Step 4: Check the console:

```
Now open the browser developer console.
Are there any errors or warnings? If so, what are they?
```

Expected Agent Actions

```
→ browser_navigate(url="http://localhost:8000")
→ browser_snapshot() – page structure captured
→ browser_console_messages()
   [WARNING] Deprecated API used on line 42
   [ERROR] Failed to load resource: /api/data 404

Agent: Found a deprecated API warning and a 404 on /api/data
```

Exercise 3.2 — Steps 5–6

Step 5: Diagnose a layout issue:

```
The login button is partially hidden on mobile sizes.
Use the browser tool to check what's happening.
```

Step 6: Extract data from a page:

```
Go to https://example.com/pricing
Extract all pricing plan names and their monthly costs into a table.
```

Browser Tool Limitations

Limitation	Workaround
Cannot log in to sites	Provide login instructions or session cookies
JavaScript-heavy sites may load slowly	Add wait instructions
Rate limits on some sites	Space out requests
Cannot upload files	Not supported yet

Success Criteria: Opened URL · Read content · Checked console · Extracted data

Lesson 3.3

Terminal Tool

Concept · 8 min · Exercise · 12 min

What the Terminal Tool Can Do

- Run any shell command (with approval)
- See stdout, stderr, exit codes
- Read command output as context for next actions
- Chain commands based on previous results

Terminal Tool Flow

```
User: "Run the tests and fix any failures"
  → Agent: Execute "pytest tests/"
  → Result: Exit code 1, "2 failed, 5 passed"
  → Agent: Reads failures, fixes code
  → Agent: Reruns tests
  → Agent: "All tests passed."
```

Exercise 3.3 — Steps 1–2

Step 1: Check your environment:

```
Run these commands and tell me what versions we're using:
- python --version
- node --version (if applicable)
- git --version
```

Step 2: Run the test suite:

```
Run the test suite. Show me which tests pass and which fail.
```

Exercise 3.3 — Steps 3–4

Step 3: Diagnose failures:

```
One or more tests failed. What's causing these failures?
Look at the specific error messages.
```

Step 4: Fix and verify:

```
Based on your diagnosis, fix the failing tests.
Show me what you're changing before you run again.
```

Exercise 3.3 — Debug Workflow (Step 5)

```
I found a bug where the app crashes when input is empty.

1. First, run the app to reproduce the crash
2. Then, add logging to understand why
3. Finally, fix the bug and verify it works
```

```
→ python app.py → IndexError: list index out of range (line 23)
→ Agent adds guard condition
→ python app.py --test-empty-input → "No data provided"
```

Exercise 3.3 — Step 6 & Safety Rules

Step 6: React to long-running commands:

```
Run npm install or pip install. Watch the output.
If there's a warning about deprecated packages, note it and suggest fixes.
```

Approval Required	No Approval Needed
Writes files, `sudo`, `git push --force`	Version checks, `cat`, `ls`
`npm install -g`, start servers	`pytest`, `npm test`, `git status`

Success Criteria: Ran tests · Diagnosed failure · Fixed code · Verified fix

Lesson 3.4

Effective Prompting in Practice

Concept · 10 min · Exercise · 12 min

Anatomy of an Effective Prompt

1. **ROLE / CONTEXT** — "You are a senior Python developer…"
2. **TASK** — "Fix the bug in calculate_total()…"
3. **CONSTRAINTS** — "Do not change the function signature…"
4. **OUTPUT FORMAT** — "Show me the diff and explain your change…"
5. **SUCCESS CRITERIA** — "Function should return 0 for empty input…"

Bad Prompts vs. Good Prompts

Bad Prompt	Good Prompt
"Fix this code"	"Fix the IndexError in process_list() when list is empty. Do not change return type."
"Add logging"	"Add INFO-level logging to calculate() using existing logger config."
"Make it faster"	"Optimize find_user() from O(n²) to O(n log n). Don't change signature."
"Review my code"	"Review auth.py for SQL injection, password handling, session issues. Ignore style."

The "Boundaries" Technique

Always tell the agent what **NOT** to touch:

```
BOUNDARIES:
- Do NOT change: function signatures, return types, existing tests
- Do NOT touch: config files, database schemas, other modules
- Do NOT delete: comments, logging, error handling
- Do NOT add: new dependencies, external APIs, global state

Change ONLY: the function body of calculate_total()
```

Avoiding Scope Creep

The problem:

```
User: "Fix the login bug."
Agent: "Fixed login. Also refactored auth, added OAuth, reorganized codebase."
User: "Wait, I just wanted the login bug fixed!"
```

Technique	Example
Explicit boundaries	"Change ONLY login.js lines 42–56"
One thing at a time	"First, just identify the issue. Don't fix yet."
Ask for plan first	"Plan Mode: Show me what you'll change before doing it"
Use checkpoints	Create checkpoint before complex requests
Prefer diffs	"Show me the diff, don't replace the whole file"

Exercise 3.4 — Step 1: Constrained Prompt

```
Task: Fix the bug where get_user_email(user_id) returns None for valid users.

Constraints:
- Do NOT change the function signature
- Do NOT add new imports
- Do NOT modify other functions
- Do NOT add print statements (use existing logger)

Output format: Show exact diff and explain root cause.
Success criteria: Function returns email string for valid user IDs.
```

Exercise 3.4 — Steps 2–3

Step 2: Compare constrained vs. unconstrained:

```
Fix get_user_email - it's returning None sometimes.
```

Note the difference in scope of the response.

Step 3: Plan first:

```
Before making any changes, answer:
1. What files would need to change?
2. What is the root cause you suspect?
3. What are the risks?
4. Are there alternative approaches?

I will review before approving any code changes.
```

Exercise 3.4 — Steps 4–5

Step 4: Negative constraints:

```
Add error handling to the file parser.

But DO NOT:
- Change the return type (must remain dict)
- Add external dependencies
- Swallow exceptions silently (always log them)
- Change the existing test file
```

Step 5: One change at a time:

```
First, add input validation. Just show me what you'd add — don't modify yet.
[Review] Now add the validation. Show the diff before I accept.
```

Exercise 3.4 — Step 6: Prompt Templates

Save as `.cursor/prompt-templates.md`:

```
## Bug Fix Template
Task: [Describe bug]  File: [path]  Lines: [range]
Constraints: Do NOT change [signatures, imports]
Success criteria: [How to verify]

## Feature Addition Template
Plan first: Yes/No  Constraints: Do NOT break [existing functionality]

## Code Review Template
Focus: [Security, Performance, Edge cases]
Ignore: [Style, formatting]
```

Success Criteria: Constrained prompt · Plan first · Negative constraints · Template created

Module Summary

Lesson	Topic	Key Takeaway
3.1	Ask vs Agent Mode	Use Ask for questions, Agent for action
3.2	Browser Tool	Agent can see live pages and console
3.3	Terminal Tool	Agent can run commands and react
3.4	Effective Prompting	Boundaries prevent scope creep

Quick Reference Card

```
MODES:
  ASK MODE    → Read-only | Questions, learning
  AGENT MODE  → Full tools | Implementation, debugging

TOOLS:
  BROWSER  → Navigate, read, console, screenshot
  TERMINAL → Execute commands, read output
  FILE     → Read, edit, create, delete
  SEARCH   → Code search, symbol lookup

PROMPTING:
  • Be specific  • Define boundaries  • Plan first for complex tasks
  • Specify output format  • Define success criteria
```

Up Next: Module 4

Multi-Model Configuration

> Now that you understand agent modes and core tools, **Module 4** covers using different models for different tasks, managing API keys, and optimizing cost vs. quality.

End of Module 3